

Desenvolvimento de uma Ferramenta de Apoio à Decisão para Investidores com Foco em Dividendos

Gustavo Ferreira Lima

RESUMO

Este relatório apresenta a implementação da "Bússola de Valor", uma ferramenta de apoio à decisão para investidores focados em renda por dividendos. O projeto realiza a coleta e padronização de dados, a modelagem do Data Warehouse local em DuckDB, o cálculo de pontuações fundamentalistas e o desenvolvimento de um dashboard interativo em Streamlit. Foram gerados artefatos (ETL, datasets e tabelas materializadas) e validada a operação local com baixo custo operacional; o DW é atualizado automaticamente via workflow do GitHub Actions.

Palavras-chave

dividendos, duckdb, streamlit

I. INTRODUÇÃO

1. Contexto

Este projeto origina-se de uma experiência pessoal no mercado de ações, iniciada em 2019. O período, marcado por alta volatilidade e pela crise da pandemia, expôs a necessidade de um método de análise de investimentos que fosse ao mesmo tempo robusto e sistemático. A rotina de avaliação manual, empresa por empresa, consolidada em planilhas do Excel de forma semanal ou mensal, era um processo laborioso e suscetível a erros. A "Bússola de Valor" nasce, portanto, da visão de automatizar e escalar essa análise, transformando um trabalho manual exaustivo em um processo eficiente que entrega dados atualizados e insights valiosos diariamente, logo após o fechamento do mercado.

Nesse cenário, a análise fundamentalista se apresenta como uma abordagem consolidada para investidores que buscam retorno consistente e segurança na preservação de capital. Este trabalho detalha a implementação e avaliação da "Bússola de Valor", uma ferramenta de apoio à decisão para investidores focados em renda por dividendos. A solução consiste em um dashboard interativo desenvolvido em Python com Streamlit que agrega, processa e pontua (scoring) ações negociadas na B3, combinando métricas de dividendos, valuation, saúde financeira, crescimento, liquidez e volatilidade.

2. Objetivo

O objetivo central deste trabalho é desenvolver uma aplicação de apoio à decisão de ponta a ponta, capaz de:

- Coletar e consolidar dados financeiros de múltiplas fontes de forma automatizada.
- Calcular um score proprietário, fundamentado em critérios econômicos e de investimento consagrados.
- Apresentar visualmente rankings, detalhes por ativo e avaliações por setor/subsetor em uma interface interativa e intuitiva.

3. Motivação

A motivação para o desenvolvimento da "Bússola de Valor" é multifacetada, combinando um propósito prático com um desafio técnico e uma filosofia de preservação de capital:

- **Auxiliar investidores pessoa física:** O objetivo é identificar ações que combinam boa geração de dividendos com fundamentos robustos, apoiando a alocação de portfólios orientados à renda de forma sistemática e eficiente.
- **Evitar perdas de capital:** Uma motivação central é criar um "filtro de qualidade" que ajude o investidor a evitar empresas com indicadores péssimos, minimizando ao máximo o risco de perda de capital, um princípio fundamental do *value investing*.
- **Integrar conhecimento clássico e tecnologia:** O projeto busca unir os conceitos de investidores consagrados, como Décio Bazin, Benjamin Graham, Warren Buffett e Luiz Barsi, com práticas modernas de engenharia de dados, como pipelines de ETL, data warehousing e dashboards interativos.

É crucial ressaltar que a "Bússola de Valor" **não é uma recomendação de compra ou venda**. Ela funciona como uma ferramenta de apoio para análise e estudo, projetada para otimizar o tempo do investidor e fornecer uma base quantitativa sólida para que ele possa tomar suas próprias decisões de investimento com mais segurança e informação.

4. Materiais

Este trabalho foi realizado utilizando os seguintes recursos tecnológicos e bibliotecas de software:

- **Linguagem:** Python 3.13+
- **Bibliotecas principais:** pandas, numpy, plotly, streamlit, yfinance, joblib, python-dotenv, ta, tqdm, deep_translator.
- **Infraestrutura de dados:** Arquitetura de dados em camadas com arquivos Parquet para as zonas Land (duckdb/land_dw/) e Trusted (duckdb/trusted_dw/), e um Data Warehouse local (duckdb/banco_dw/dw.duckdb) em DuckDB.
- **Ferramentas de desenvolvimento:** Git/GitHub para versionamento de código e automação.
- **Scripts de ETL:** Pipeline de dados modularizado e orquestrado, presente no diretório data_engineer/.

5. Metodologia do Trabalho

O desenvolvimento do projeto seguiu uma metodologia estruturada, compreendendo as seguintes etapas:

1. **Coleta e Padronização de Dados:** Levantamento de dados públicos de APIs (yfinance, brapi) e arquivos históricos, seguido de um processo de normalização para garantir a consistência.
2. **ETL (Extract, Transform, Load):** Construção de um pipeline de dados em camadas (Land, Trusted, DW), que transforma os dados brutos em um data warehouse analítico, coeso e performático.
3. **Cálculo de Score:** Implementação de um sistema de pontuação baseado em regras objetivas, que atribui pontos (até 1000) por critério, como Dividend Yield, Valuation, ROE, Payout, Endividamento, Crescimento e Margem de Graham.
4. **Desenvolvimento do Dashboard:** Criação de uma interface interativa em Streamlit para permitir a filtragem dinâmica, a visualização de rankings, a análise detalhada por ativo e a exploração de painéis setoriais.
5. **Validação:** Realização de testes manuais e execução local do pipeline completo para verificar a coerência entre os dados de entrada e as métricas exibidas no dashboard.

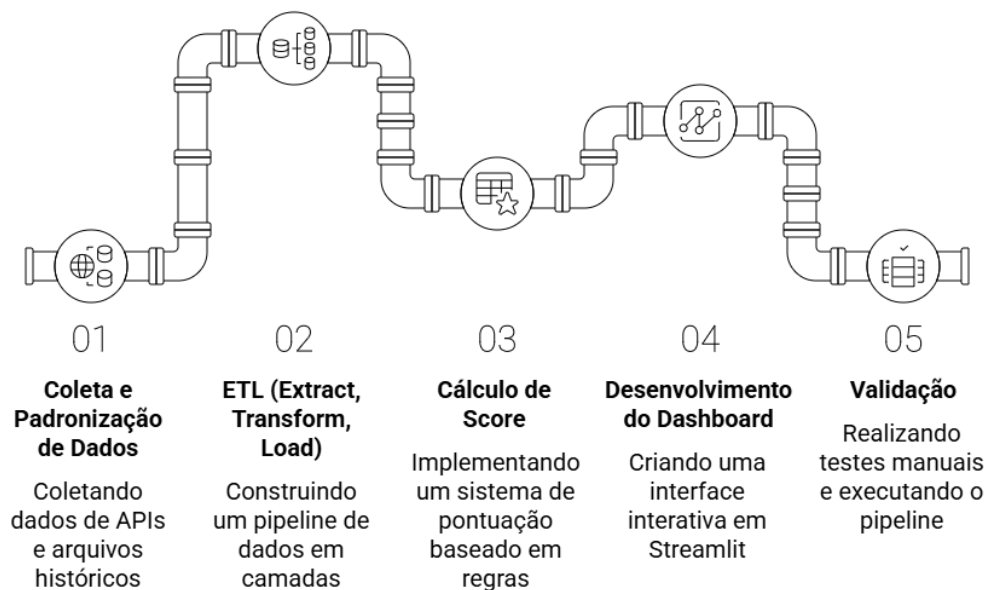


Figura 1. Pipeline de dados

II. REVISÃO DE TECNOLOGIAS E PRÁTICAS

1. COLETA DE DADOS E APIS

Para construir uma base de dados abrangente, o sistema integra dados de múltiplas fontes, cada uma com uma finalidade específica:

- yfinance: Utilizada para a extração de séries históricas de preços e dividendos, fundamentais para cálculos de retorno e volatilidade.
- brapi: Empregada para enriquecer a base de dados com atributos cadastrais das empresas, como setor de atuação, e outros indicadores financeiros relevantes.

2. ETL E ARMAZENAMENTO

A estratégia de ETL e armazenamento foi um pilar central do projeto, garantindo eficiência e baixo custo operacional:

- Os scripts no diretório `data_engineer/` seguem uma ordem numérica que assegura a correta execução das dependências entre as etapas do pipeline (extração → transformação → cálculo de indicadores → scoring).
- O formato de persistência adota uma abordagem híbrida: arquivos Parquet são utilizados para as camadas de dados intermediárias (Land e Trusted), beneficiando-se da compressão e performance em leituras colunares, enquanto o Data Warehouse final é materializado em um único arquivo DuckDB (`duckdb/banco_dw/dw.duckdb`). A escolha pelo DuckDB foi estratégica, pois, por ser um banco de dados analítico embutido, dispensa a necessidade de um servidor dedicado, reduzindo drasticamente os custos e a complexidade de infraestrutura em comparação com soluções como MySQL ou SQL Server.

3. ANÁLISE E MODELAGEM

A modelagem do score é o núcleo intelectual do projeto, traduzindo a filosofia de investimentos em um sistema de pontuação quantitativo:

- O modelo é uma combinação de regras financeiras consagradas pela literatura e heurísticas pragmáticas observadas no mercado. A pontuação é modular e compreende componentes para dividendos, valuation, rentabilidade, saúde financeira, crescimento, volatilidade e liquidez, permitindo uma análise multifacetada de cada ativo.

4. VISUALIZAÇÃO E INTERAÇÃO

A camada de visualização foi construída com o objetivo de ser ao mesmo tempo poderosa e intuitiva:

- A combinação de Streamlit e Plotly permitiu a criação de uma interface interativa e responsiva. A estrutura da aplicação é modular, com componentes dedicados para a renderização dos filtros na barra lateral (`app/components/filters.py`) e para a organização do conteúdo principal em abas (`app/components/tabs_layout.py`), facilitando a manutenção e a expansão futura da ferramenta.

III. DESENVOLVIMENTO

1. Arquitetura do Sistema

A arquitetura da solução foi projetada em camadas para garantir um fluxo de dados desacoplado, manutenível e performático, desde a coleta até a visualização. O sistema é composto por três componentes principais: o pipeline de engenharia de dados, o data warehouse e a aplicação de visualização.

- **data_engineer/ (Camada de Extração):** Este diretório contém os scripts responsáveis pela extração de dados brutos de fontes externas, como as APIs yfinance e brapi. Os scripts são numerados para garantir a ordem de execução e orquestrados pelo loader.py. A saída desta camada são arquivos em formato Parquet, que são armazenados na camada *Land* do data lake.
- **duckdb/ (Camada de Armazenamento e Transformação):** Centraliza toda a lógica de armazenamento e processamento de dados. É subdividida em:
 - **land_dw/:** A camada *Land*, que armazena os dados brutos extraídos, servindo como uma fonte de verdade inicial e permitindo o reprocessamento sem a necessidade de consultar as APIs novamente.
 - **trusted_dw/:** A camada *Trusted*, onde os dados da camada *Land* são limpos, transformados, enriquecidos com cálculos de indicadores e onde a lógica de *scoring* é aplicada. O resultado é um conjunto de dados coeso e pronto para análise.
 - **banco_dw/:** Contém o Data Warehouse (DW) final, materializado no arquivo dw.duckdb. Este arquivo único consolida os dados da camada *Trusted* em um formato analítico otimizado para as consultas da aplicação.
 - **carga/:** Orquestra o processo de carga da camada *Land* para a *Trusted* e, subsequentemente, para o Data Warehouse final.
- **app/ (Camada de Apresentação):** Contém a aplicação web interativa desenvolvida com Streamlit.
 - app.py: Ponto de entrada da aplicação, responsável pela estrutura da interface e orquestração dos componentes visuais.
 - data_loader.py: Módulo que realiza a conexão com o dw.duckdb e encapsula as consultas SQL necessárias para carregar os dados para a interface.
 - components/: Módulos reutilizáveis que renderizam partes específicas da UI, como os filtros da barra lateral e as abas de conteúdo.

Fluxo de Dados:

O fluxo de dados segue um padrão ELT (Extract, Load, Transform) moderno e robusto:

1. **Extração:** O run.py inicia o processo, acionando os scripts em data_engineer/ que coletam os dados brutos e os depositam na camada land_dw em formato Parquet.
2. **Carga e Transformação:** Em seguida, os scripts em duckdb/carga/ são executados. Eles carregam os dados brutos da camada land_dw, aplicam as transformações (limpeza, cálculos, scoring) e salvam o resultado na camada trusted_dw, também em Parquet.
3. **Materialização do DW:** Os dados da camada trusted_dw são carregados e consolidados no Data Warehouse analítico, o arquivo dw.duckdb.
4. **Visualização:** O usuário interage com a aplicação Streamlit. O app/data_loader.py submete consultas SQL ao dw.duckdb para buscar os dados necessários, que são então exibidos nos gráficos e tabelas da interface. Este processo é automatizado e executado diariamente através de um workflow do GitHub Actions.

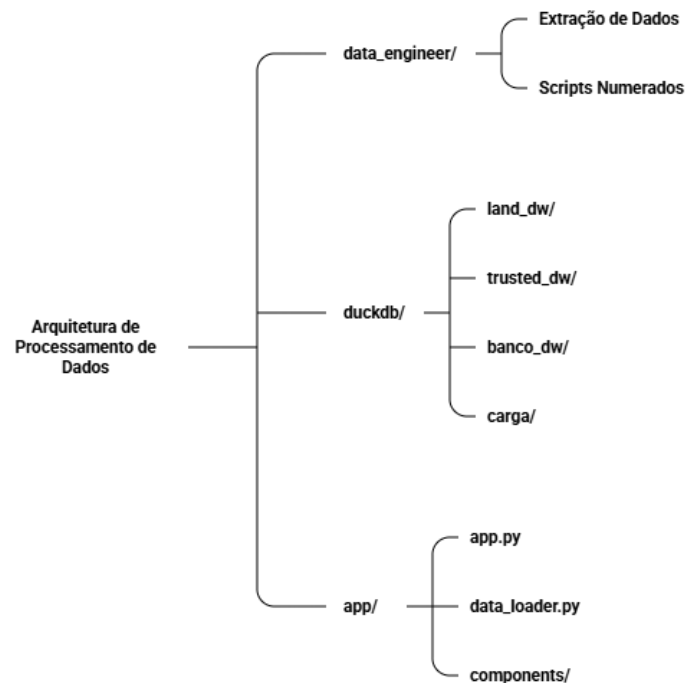


Figura 2. Arquitetura e Processamento de Dados

2. Lógica de Scoring

O sistema de pontuação foi desenhado para refletir uma filosofia de investimento balanceada, traduzindo critérios fundamentalistas em um modelo quantitativo. A pontuação máxima teórica é de **1000 pontos**, distribuídos em categorias que avaliam diferentes dimensões da saúde financeira e do potencial de uma empresa. A soma dos pontos máximos de cada categoria totaliza exatamente 1000, conforme detalhado abaixo:

- **Dividendos (200 pts):** Critério com maior peso, focado na consistência da distribuição de proventos. A pontuação é uma média ponderada, onde o Dividend Yield (DY) médio dos últimos 5 anos tem maior relevância que o DY dos últimos 12 meses.
- **Valuation (180 pts):** Avalia se o preço atual da ação está descontado em relação aos seus fundamentos. A pontuação é baseada nos indicadores Preço/Lucro (P/L) e Preço/Valor Patrimonial (P/VP), favorecendo empresas com múltiplos baixos.
- **Críticos de Mercado (140 pts):** Engloba fatores de liquidez, volatilidade, capitalização e geração de caixa. A pontuação é distribuída entre a Liquidez Média Diária (até 35 pts), o Beta (volatilidade, até 35 pts), o Market Cap (até 35 pts) e o Free Cash Flow Yield (FCF Yield, até 35 pts).
- **Saúde Financeira (130 pts):** Mede a solidez financeira da companhia. Utiliza indicadores como Dívida Líquida/EBITDA e Liquidez Corrente para premiar empresas com baixo endividamento e boa capacidade de honrar suas obrigações de curto prazo.
- **Rentabilidade (110 pts):** Analisa a eficiência da gestão em gerar valor para o acionista. A pontuação é derivada do Retorno sobre o Patrimônio Líquido (ROE) e da política de Payout, com regras de pontuação ajustadas para as particularidades do setor financeiro.
- **Crescimento, Sentimento e Timing (180 pts):** Avalia a capacidade da empresa de expandir suas receitas (até 50 pts), o sentimento de analistas (até 60 pts) e o ciclo de mercado para identificar o *timing* de compra ou venda (até 70 pts).
- **Críticos de Graham (150 pts):** Incorpora a filosofia de Benjamin Graham, calculando a "margem de segurança" com base no preço justo. A pontuação pode chegar a 150 pontos para ativos com grande desconto.

A distribuição soma **1000 pontos**, fornecendo uma base clara e equilibrada para a avaliação dos ativos.

Penalidades

Para garantir que o score reflita riscos críticos, o modelo aplica penalidades que podem reduzir significativamente a pontuação final de um ativo:

- **Recuperação Judicial (Duplo Impacto):** Este critério aplica uma penalidade em dois níveis, refletindo o alto risco associado a empresas em dificuldades financeiras.
 - **Nível do Ativo (Penalidade Máxima: Score Total):** Empresas que se encontram em processo de recuperação judicial têm seu score individual **automaticamente zerado**. Esta é a penalidade mais severa, pois indica um risco existencial que invalida a maior parte das métricas fundamentalistas convencionais.
 - **Nível do Setor:** O score agregado de um setor também é penalizado com base na **quantidade de empresas em recuperação judicial** que ele contém. Um setor com um histórico de RJs é percebido como mais arriscado, e essa penalidade ajusta a avaliação coletiva para baixo, alertando o investidor sobre uma possível fragilidade estrutural ou cíclica daquele segmento.
- **Prejuízo Recorrente (Penalidade Máxima: -150 pts):** Empresas que apresentam prejuízo líquido nos últimos 12 meses ou na média dos últimos 5 anos sofrem deduções na pontuação. A penalidade é progressiva, podendo chegar a 150 pontos, sinalizando ineficiência operacional ou desafios setoriais.
- **Endividamento Elevado (Penalidade Máxima: -100 pts):** Níveis de alavancagem considerados excessivos, medidos pela relação Dívida Líquida/Patrimônio Líquido, resultam em penalidades. A dedução pode chegar a 100 pontos, refletindo um maior risco financeiro e menor flexibilidade da companhia.



Figura 3. Equilibrando Pontuação e Penalidades

3. Boas práticas implementadas

Visando a qualidade, performance e manutenibilidade do código, diversas boas práticas de engenharia de software, banco de dados e desenvolvimento Python foram aplicadas ao longo do projeto:

Boas Práticas em Banco de Dados e Arquitetura de Dados

- **Arquitetura em Camadas (Land, Trusted, DW):** A estruturação do data lake em camadas distintas (land_dw, trusted_dw) antes da carga no Data Warehouse (banco_dw) é uma prática de mercado consolidada. Ela garante a rastreabilidade, permite o reprocessamento de dados de forma isolada e desacopla as etapas de extração, transformação e consumo.
- **Uso Estratégico de DuckDB e Parquet:** A escolha do DuckDB como Data Warehouse analítico local, combinada com o uso do formato Parquet para as camadas intermediárias, otimiza a performance e reduz custos. O Parquet oferece compressão e leitura colunar eficiente, enquanto o DuckDB proporciona consultas analíticas de alta velocidade sem a necessidade de um servidor de banco de dados tradicional.
- **Estratégia de Carga Híbrida (Completa e Incremental):** O pipeline adota uma abordagem de carga mista. A maioria das tabelas é recriada a cada execução (CREATE OR REPLACE TABLE), garantindo um estado final consistente e idempotente. No entanto, para tabelas de histórico volumosas (como todos_dividendos), é utilizada uma carga incremental, onde apenas os novos registros são adicionados, otimizando significativamente o tempo de processamento e o uso de recursos.
- **Abstração de Acesso a Dados:** A centralização de todo o acesso a dados no módulo app/data_loader.py abstrai as consultas SQL do restante da aplicação. Isso desacopla a camada de visualização da camada de dados, facilitando a manutenção e futuras migrações de banco de dados.

Boas Práticas em Desenvolvimento Python

- **Modularidade e Separação de Responsabilidades:** O código foi organizado em módulos com responsabilidades bem definidas. Por exemplo, o pipeline de ETL está em data_engineer/, a lógica da aplicação em app/, e os componentes de UI em app/components/. Essa estrutura facilita a manutenção, o teste e a evolução do sistema.
- **Tratamento Robusto de Erros:** Os scripts de coleta de dados (data_engineer/08-indicadores.py) implementam blocos try...except para capturar falhas durante as chamadas de API e o processamento de dados. Isso torna o pipeline mais resiliente, permitindo que ele continue a execução mesmo que um ativo específico falhe.
- **Código Limpo e Legível:** O uso de docstrings para explicar o propósito de funções e módulos, nomes de variáveis descritivos e a utilização de pathlib para manipulação de caminhos de arquivos contribuem para um código mais limpo e de fácil compreensão.
- **Otimização de Performance na Aplicação:**
 - **Cache do Streamlit (@st.cache_data):** O uso do decorador de cache do Streamlit acelera o recarregamento da aplicação, evitando a re-execução de consultas pesadas a cada interação do usuário.
 - **Processamento Paralelo:** A implementação de joblib para o cálculo de scores em paralelo (app/scoring.py) demonstra uma preocupação com a otimização de performance em tarefas computacionalmente intensivas, embora a abordagem principal seja a carga de scores pré-calculados.

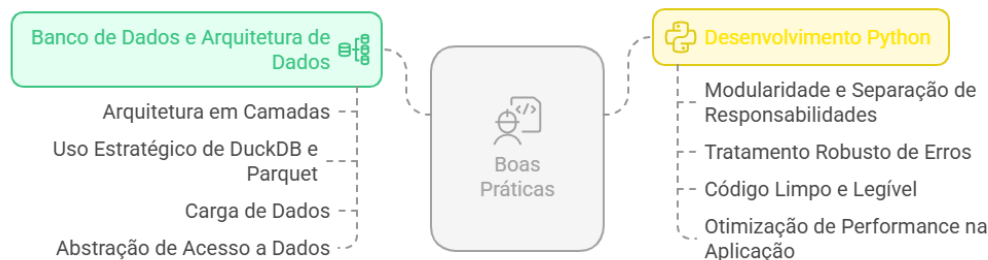


Figura 4. Boas praticas

IV. RESULTADOS E ANÁLISE

1. Artefatos gerados

A execução do pipeline de dados resulta em um conjunto de artefatos coesos e prontos para consumo:

- **O Data Warehouse (dw.duckdb):** Artefato principal do projeto, que consolida todos os dados tratados e serve como fonte única e performática para a aplicação.
- **Camadas de Dados Intermediárias:** Os arquivos Parquet nas camadas Land e Trusted garantem a reprodutibilidade e a rastreabilidade de todo o pipeline de transformação.
- **Dashboard Interativo:** A aplicação Streamlit, executável via `streamlit run app/app.py`, que apresenta os resultados de forma clara e interativa, contendo:
 - **Ranking Geral e Detalhado:** Tabelas interativas que classificam os ativos pelo Score Total (exibido como barra de progresso) e permitem a visualização de dezenas de indicadores fundamentalistas e técnicos.
 - **Análise Individual:** Uma seção dedicada que detalha a composição do Score Total de cada ativo, separando os pontos positivos das penalidades aplicadas.
 - **Análise de Dividendos:** Gráficos de série temporal dos proventos, frequência de pagamento por mês e rankings dos maiores pagadores de dividendos.
 - **Análise Setorial:** Um ranking que avalia o desempenho agregado de cada subsetor da bolsa, com sua pontuação final e gráficos comparativos.
 - **Variação de Preços:** Colunas no ranking que exibem a valorização percentual dos ativos nos últimos 1 e 6 meses.



Figura 5. Bússola de valor

2. Validação e Verificação

A garantia da acurácia, consistência e confiabilidade dos dados é um pilar fundamental da "Bússola de Valor". Para assegurar a integridade do sistema, desde a coleta inicial até a visualização final, foi implementado um processo de validação multifacetado que abrange a qualidade dos dados, a lógica de negócio e a robustez do pipeline.

- **Validação da Integridade dos Dados:** A primeira linha de defesa é a validação no nível dos dados. Durante a fase de transformação (camada *Trusted*), são aplicadas rotinas de limpeza e normalização. Isso inclui a coerção de tipos de dados para garantir consistência numérica e textual, o tratamento de valores ausentes (NaN) para evitar erros de cálculo, e a padronização de tickers entre as diferentes fontes de API (yfinance, brapi), assegurando que os dados se refiram inequivocamente ao mesmo ativo.
- **Validação da Arquitetura e do Pipeline:** A robustez do fluxo de dados é garantida pela arquitetura do pipeline. A execução sequencial e numerada dos scripts (data_engineer/) assegura que as dependências sejam respeitadas, prevenindo a execução de uma etapa antes que seus pré-requisitos sejam concluídos. A natureza da carga de dados, que utiliza a abordagem CREATE OR REPLACE TABLE para a maioria das tabelas, garante que re-execuções do pipeline resultem em um estado final consistente, eliminando riscos de duplicação de dados ou corrupção.
- **Verificação End-to-End:** Para validar a jornada completa do dado, foi conduzida uma auditoria de rastreabilidade. O processo de ETL inicia-se com o loader que deposita os dados brutos na camada de pouso (land_dw). Em seguida, os dados são tratados e enriquecidos, sendo persistidos na camada trusted_dw. Finalmente, são consolidados no Data Warehouse dw.duckdb para consumo pela aplicação. A verificação consistiu em selecionar um indicador específico no dashboard (e.g., o P/L de um ativo) e rastreá-lo no sentido inverso: do componente visual no Streamlit, passando pela consulta SQL no data_loader.py, pelo dado no dw.duckdb, pelo arquivo Parquet correspondente na trusted_dw, até chegar ao dado bruto original na land_dw. Essa auditoria de ponta a ponta confirma que cada etapa do pipeline (extração, tratamento e carga) opera como esperado, garantindo que a informação exibida ao usuário final é um reflexo fiel e acurado do dado de origem após todas as transformações aplicadas.

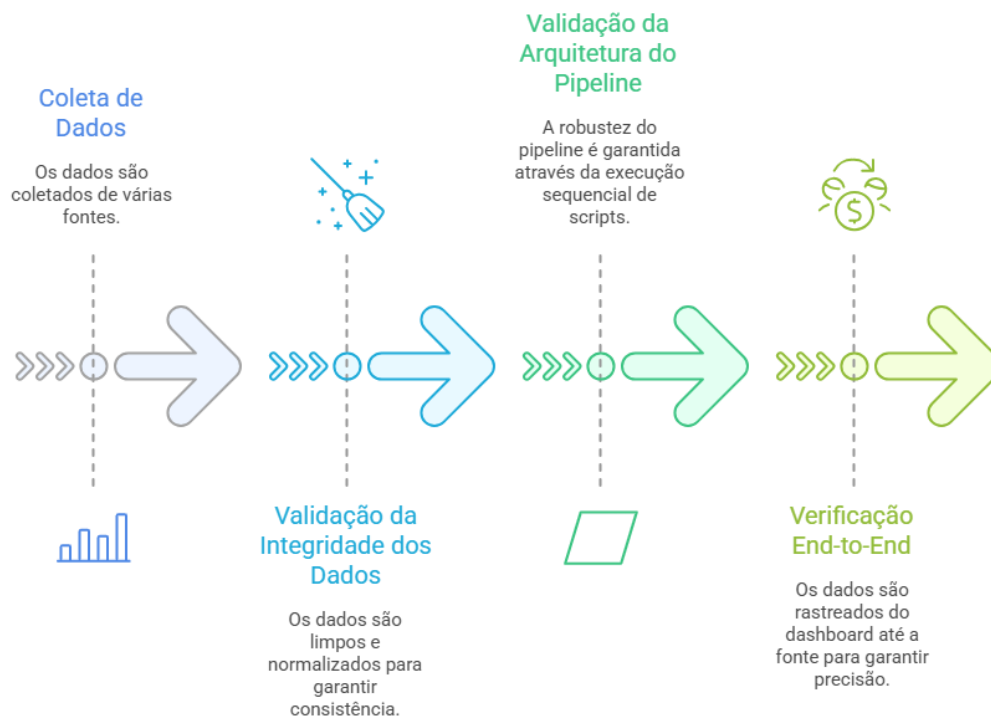


Figura 6. Validação e verificação dos dados

V. CONCLUSÃO

O projeto "Bússola de Valor" entrega uma ferramenta operacional de alto valor, que combina com sucesso práticas clássicas de análise fundamentalista com técnicas modernas de engenharia de dados e visualização interativa. A solução final facilita a comparação objetiva entre empresas da B3 com foco em geração de dividendos e permite ao investidor investigar as justificativas de cada pontuação, promovendo uma tomada de decisão mais informada e ágil.

1. Dificuldades Encontradas

O desenvolvimento de um projeto engenharia de dados com múltiplas integrações apresentou desafios técnicos e conceituais significativos. A superação destes obstáculos foi crucial para o sucesso da ferramenta e envolveu as seguintes frentes:

- **Normalização de Dados:** Um dos maiores desafios foi a normalização dos diversos formatos de dados provenientes das diferentes APIs e fontes, exigindo um tratamento cuidadoso para unificar as informações de forma consistente.
- **Tratamento de Valores Faltantes:** A ausência de dados em algumas fontes para determinados tickers demandou a implementação de lógicas de tratamento para evitar erros e garantir que a pontuação não fosse indevidamente prejudicada.
- **Definição do Modelo de Scoring:** A definição consensual dos limites e pesos de cada critério do score foi um processo iterativo, que exigiu uma combinação de pesquisa em literatura financeira e testes empíricos para alcançar um modelo balanceado e representativo.

2. Aplicabilidade do Trabalho

O projeto "Bússola de Valor" possui uma dupla aplicabilidade. Primeiramente, representa uma significativa contribuição acadêmica ao documentar de ponta a ponta a construção de uma solução de engenharia de dados aplicada ao mercado financeiro, desde a coleta e estruturação de dados até a criação de um modelo de scoring e sua disponibilização em um dashboard interativo.

O trabalho serve como um estudo de caso prático sobre a aplicação de tecnologias como DuckDB e Streamlit para resolver problemas complexos de análise de investimentos.

Em segundo lugar, a ferramenta foi concebida para ser uma solução robusta e acessível a todos os investidores. Seu propósito é democratizar o acesso à análise fundamentalista de qualidade, oferecendo uma plataforma que automatiza a coleta de dados e a aplicação de critérios de avaliação consagrados. Dessa forma, a "Bússola de Valor" se posiciona como um recurso educacional e de apoio à decisão, capacitando o investidor a aprimorar suas próprias análises e a construir portfólios de forma mais informada e sistemática.

VI. BIBLIOGRAFIA

- ARUN, Ran. *yfinance*. Disponível em: <https://pypi.org/project/yfinance/>. Acesso em: 8 nov. 2025.
- BARSI FILHO, Luiz. *O rei dos dividendos: a saga do filho de imigrantes pobres que se tornou o maior investidor pessoa física da bolsa de valores brasileira*. 1. ed. Rio de Janeiro: Sextante, 2022.
- BAZIN, Décio. *Faça fortuna com ações antes que seja tarde*. 3. ed. São Paulo: CLA, 2005.
- DUCKDB. *DuckDB Documentation*. Disponível em: <https://duckdb.org/docs/>. Acesso em: 8 nov. 2025.
- FEUERSTEIN, Steven. *Oracle PL/SQL programming*. 6. ed. Sebastopol: O'Reilly Media, 2014.
- FISHER, Philip A. *Ações comuns, lucros extraordinários*. 1. ed. São Paulo: Benvirá, 2011.
- GRAHAM, Benjamin. *O investidor inteligente*. 4. ed. Rio de Janeiro: HarperCollins, 2022.
- KIYOSAKI, Robert T. *Pai rico, pai pobre*. 20. ed. aniv. Rio de Janeiro: Alta Books, 2017.
- KNAFLIC, Cole Nussbaumer. *Storytelling com dados: um guia sobre visualização de dados para profissionais de negócios*. 1. ed. Rio de Janeiro: Alta Books, 2017.
- KYTE, Tom. *Expert Oracle database architecture*. 3. ed. New York: Apress, 2013.
- LONEY, Kevin. *Oracle Database 12c: the complete reference*. 1. ed. New York: McGraw-Hill Education, 2013.
- LUEDERS, Anderson; PORA, Ib. *Investindo em small caps: um roteiro completo para se tornar um investidor de sucesso*. 1. ed. São Paulo: DVS Editora, 2014.
- MICROSOFT. *Visual Studio Code Docs*. Disponível em: <https://code.visualstudio.com/docs>. Acesso em: 8 nov. 2025.
- NAPKIN.AI. *Napkin.ai*. Disponível em: <https://www.napkin.ai/>. Acesso em: 8 nov. 2025.
- NIELD, Thomas. *Introdução à linguagem SQL*. 1. ed. Sebastopol: O'Reilly Media, 2016.
- PANDAS. *pandas Documentation*. Disponível em: <https://pandas.pydata.org/docs/>. Acesso em: 8 nov. 2025.
- PLOTLY. *Plotly Python Graphing Library*. Disponível em: <https://plotly.com/python/>. Acesso em: 8 nov. 2025.
- STREAMLIT. *Streamlit Documentation*. Disponível em: <https://docs.streamlit.io>. Acesso em: 8 nov. 2025.